Laboratorio 8 Servicios web con Django

Repositorio del proyecto: Proyecto_Django_Suarez

Desarrollo de Aplicaciones Empresariales · Django REST Framework

En esta práctica expuse un modelo de evaluaciones mediante una API REST. Organicé los datos como examen, pregunta y opción, y definí cómo se representan sus relaciones para que el cliente pueda enviar y entender el contenido sin construir JSON a mano en cada vista.

Diseño y validación

Los tres recursos usan serializadores de modelo y conjuntos de vistas registrados con un enrutador. Las opciones viajan anidadas dentro de una pregunta; el examen se identifica por su ID. La creación y actualización de preguntas son transaccionales para no dejar opciones a medio guardar.

El serializador rechaza una pregunta si no tiene exactamente una opción correcta. En la prueba provoqué los casos de ninguna respuesta correcta y de dos respuestas correctas; ambos devolvieron HTTP 400.

Rutas y uso

La API publica exámenes en `/api/exams/`, preguntas en `/api/questions/` y opciones en `/api/options/`. Los conjuntos de vistas permiten listar, consultar detalle, crear, modificar y eliminar recursos.

El listado de preguntas permite filtrar por examen y buscar por texto. La paginación entrega diez resultados por página. La tabla adjunta resume los contratos principales para que quien consuma el servicio sepa qué enviar y qué esperar.

Verificación

Las cuatro pruebas HTTP pasaron. Cubren lectura y operaciones de creación, edición y borrado; verifican las opciones anidadas, el filtro y la búsqueda; y confirman los errores de validación. `manage.py check` no reportó problemas.

La configuración inicial usa `AllowAny`, tal como indica la guía de partida. Antes de exponer una API fuera del entorno académico, debe cambiarse por autenticación y permisos que correspondan al uso real.

Conclusión

Separar modelos, serializadores, vistas y rutas dejó cada responsabilidad fácil de revisar. La validación de la respuesta correcta protege una regla del dominio que, de otro modo, podría romperse con una petición HTTP incompleta o mal formada.

Recurso	Ruta	Solicitud de ejemplo	Respuesta esperada
Exámenes	`/api/exams/`	POST `{'title':'Evaluación'}`	201; listado paginado
Preguntas	`/api/questions/`	POST con `exam`, `text`, `position` y dos `options`	201; opciones anidadas
Preguntas	`/api/questions/?exam=1&search=capital`	GET	200; resultados filtrados
Opciones	`/api/options/`	POST con `question`, `text`, `is_correct`	201; opción creada
Validación	`/api/questions/`	POST con 0 o 2 opciones correctas	400; mensaje de validación

Modelo y contrato de los recursos

La API se construyó alrededor de tres recursos: un examen contiene preguntas y una pregunta contiene opciones. El lado servidor conserva las relaciones como claves foráneas; el cliente identifica el examen con un entero y puede enviar las opciones juntas al crear o editar una pregunta.

```
Exam 1 ──────── * Question 1 ──────── * Option
                     (una opción debe tener is_correct=true)
```

Serializadores y validación propia

```
class QuestionSerializer(serializers.ModelSerializer):
    options = NestedOptionSerializer(many=True)
    exam = serializers.PrimaryKeyRelatedField(
        queryset=Exam.objects.all())

    def validate_options(self, options):
        correct = [item for item in options
                   if item.get("is_correct", False)]
        if len(correct) != 1:
            raise serializers.ValidationError(
                "La pregunta debe tener exactamente una opción correcta.")
        return options

    @transaction.atomic
    def create(self, validated_data):
        options = validated_data.pop("options")
        question = Question.objects.create(**validated_data)
        Option.objects.bulk_create([
            Option(question=question, **item) for item in options])
        return question
```

La validación vive en el serializador porque pertenece a la entrada HTTP. La transacción mantiene atómica la creación: o se guarda la pregunta con todas sus opciones, o no se guarda nada. La actualización reemplaza las opciones solo cuando el cliente envía el campo `options`; un PATCH de otro campo no las borra.

Vistas, rutas, búsqueda y paginación

```
class QuestionViewSet(viewsets.ModelViewSet):
    serializer_class = QuestionSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ["text"]

    def get_queryset(self):
        queryset = (Question.objects.select_related("exam")
                    .prefetch_related("options")
                    .order_by("position", "id"))
        exam_id = self.request.query_params.get("exam")
        if exam_id:
            queryset = queryset.filter(exam_id=exam_id)
        return queryset

router.register("exams", ExamViewSet, basename="exam")
router.register("questions", QuestionViewSet, basename="question")
router.register("options", OptionViewSet, basename="option")
```

`ModelViewSet` reúne las acciones REST habituales y el enrutador genera las rutas para listado y detalle. En preguntas, `exam` filtra por evaluación y `search` busca en el enunciado. La paginación global devuelve diez elementos por página con `count`, `next`, `previous` y `results`.

Ejemplos de peticiones y respuestas

```
POST /api/questions/
Content-Type: application/json

{
  "exam": 1,
  "text": "¿Cuánto es 2 + 2?",
  "position": 1,
  "options": [
    {"text": "4", "is_correct": true},
    {"text": "5", "is_correct": false}
  ]
}

GET /api/questions/?exam=1&search=2%20%2B%202
```

Operación	Ruta	Estado esperado
Listar o ver detalle	GET `/api/exams/`, `/api/questions/`, `/api/options/`	200 OK
Crear recurso	POST a colección	201 Created
Reemplazar o modificar	PUT o PATCH a detalle	200 OK
Eliminar	DELETE a detalle	204 No Content
Campo obligatorio ausente o FK inexistente	POST con JSON inválido	400 Bad Request
Cero o más de una opción correcta	POST `/api/questions/`	400 Bad Request

Pruebas ejecutadas

Prueba	Evidencia
Examen	GET de colección, creación, detalle, PATCH y DELETE.
Pregunta	Filtro por examen, búsqueda, opciones anidadas y creación.
Validación	0 y 2 opciones correctas responden 400.
Opción	GET de colección, POST, PATCH y DELETE.

Las cuatro pruebas HTTP pasaron y `manage.py check` no reportó problemas. El test comprueba los códigos de estado y los datos JSON usando el cliente de pruebas de Django REST Framework. Para la evidencia pedida por el curso, también se puede abrir `/api/` en el navegador y repetir las mismas operaciones con un cliente HTTP externo.

Instalación, alcance y conclusión

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py test
python manage.py runserver
```

La API queda lista para el entorno del laboratorio. Mantiene `AllowAny` porque esa es la configuración inicial de la guía; no se debe exponer públicamente con ese permiso. Antes de un despliegue real, hace falta activar autenticación y limitar quién puede crear o borrar evaluaciones. Separar serializadores, vistas y rutas dejó clara la responsabilidad de cada pieza, y la validación protege una regla que el cliente no debería poder omitir.